

پروژه پاتوق (پاتوق)
سامانه مدیریت و رزرو کافه و رستوران

گزارش اسپرینت سوم
تکمیل، امنیت و رفع باگ

درس: طراحی نرم افزار
مدت اسپرینت: ۲ هفته
تاریخ تهیه: تیر ۱۴۰۵

این گزارش پوشش دهی اسپرینت سوم پروژه پاتوق را شرح می دهد که در آن امنیت سیستم تقویت شد، باگ های شناسایی شده رفع گردیدند و صفحات ناقص تکمیل شدند.

فهرست مطالب

۲	۱ هدف اسپرینت
۲	۱.۱ اهداف اصلی
۳	۲ Sprint Backlog
۴	۳ چرخش توکن تازه سازی
۴	۱.۳ مشکل
۴	۲.۳ راه حل: Rotation Token
۴	۳.۳ پیاده سازی
۵	۴ محدودیت نرخ درخواست ها
۵	۱.۴ پیاده سازی
۵	۲.۴ رفتار در صورت تجاوز
۵	۳.۴ هدف امنیتی
۶	۵ رفع مشکل CORS برای پورت ۵۱۷۴
۶	۱.۵ مشکل
۶	۲.۵ راه حل
۷	۶ استاندارد سازی پاسخ خطا با RFC ۷۸۰۷
۷	۱.۶ مشکل
۷	۲.۶ راه حل
۷	۳.۶ کدهای وضعیت استاندارد
۸	۷ بهبود امنیت OTP
۸	۱.۷ یکبار مصرف بودن OTP
۸	۲.۷ حذف OTP از لاگ
۹	۸ رفع باگ های فرانت اند
۹	۱.۸ باگ ۱: مدیریت خطای ۴۰۱ در client.ts
۹	۲.۸ باگ ۲: پیام های خطای انگلیسی
۹	۳.۸ باگ ۳: تقویم شمسی در فرم رزرو
۹	۴.۸ باگ ۴: HasIncompleteProfile (تغییر نام IsNewUser)
۱۰	۹ بهبود امنیت ورود مدیر ارشد
۱۰	۱.۹ مشکل
۱۰	۲.۹ راه حل
۱۱	۱۰ صفحات تکمیل شده در این اسپرینت
۱۱	۱.۱۰ صفحه تکمیل پروفایل (CompleteProfile)
۱۱	۲.۱۰ پروفایل مشتری (Profile)
۱۱	۳.۱۰ پروفایل مدیر رستوران (ManagerProfile)
۱۱	۴.۱۰ مدیریت میز (TableManagement)

۱۲	۱۱	تست نهایی و استقرار
۱۲	۱.۱۱	تست‌های انجام‌شده
۱۲	۲.۱۱	محیط استقرار
۱۳	۱۲	بازبینی و گذر اسپرینت
۱۳	۱.۱۲	Review Sprint
۱۳	۲.۱۲	Retrospective Sprint
۱۳	۳.۱۲	آمار کلی پروژه

۱ هدف اسپرینت

اسپرینت سوم با هدف تکمیل سیستم، بهبود امنیت و رفع تمامی باگ‌های شناسایی شده در اسپرینت‌های قبل آغاز شد. این اسپرینت آخرین اسپرینت پروژه است و در پایان باید سیستم آماده تحویل نهایی باشد.

۱.۱ اهداف اصلی

- پیاده‌سازی چرخش توکن تازه‌سازی (Refresh Token Rotation)
- اضافه کردن محدودیت نرخ (Rate Limiting) برای Endpointهای احراز هویت
- رفع مشکل CORS برای پورت 5174
- استانداردسازی فرمت پاسخ‌های خطا (RFC 7807)
- بهبود امنیت OTP
- رفع باگ مدیریت خطای ورود در کلاینت
- تکمیل صفحه CompleteProfile
- رفع باگ تقویم شمسی
- بهبود امنیت ورود ادمین
- تکمیل صفحات پروفایل

Sprint Backlog ۲

وضعیت	مسئول	نوع	آیتم
انجام شد	بک‌اند	امنیت	چرخش توکن تازه‌سازی
انجام شد	بک‌اند	امنیت	محدودیت نرخ احراز هویت
انجام شد	بک‌اند	امنیت	محدودیت نرخ عمومی
انجام شد	بک‌اند	باگ	رفع مشکل CORS پورت ۵۱۷۴
انجام شد	بک‌اند	بهبود	فرمت خطا RFC 7807
انجام شد	بک‌اند	امنیت	OTP یکبار مصرف
انجام شد	بک‌اند	امنیت	حذف OTP از لاگ
انجام شد	فرانت‌اند	باگ	باگ ورود: 401 handler
انجام شد	فرانت‌اند	بهبود	پیام‌های خطای فارسی
انجام شد	بک‌اند	بهبود	تغییر نام IsNewUser
انجام شد	فرانت‌اند	ویژگی	صفحه تکمیل پروفایل
انجام شد	فرانت‌اند	باگ	رفع باگ تقویم شمسی
انجام شد	بک‌اند+فرانت	امنیت	بهبود امنیت ورود ادمین
انجام شد	فرانت‌اند	ویژگی	تکمیل پروفایل مشتری
انجام شد	فرانت‌اند	ویژگی	تکمیل پروفایل مدیر رستوران
انجام شد	فرانت‌اند	ویژگی	مدیریت میز (تکمیل)
انجام شد	هر دو	تست	تست نهایی یکپارچگی

۳ چرخش توکن تازه‌سازی

۱.۳ مشکل

در اسپرینت اول، توکن‌های تازه‌سازی پس از استفاده باطل نمی‌شدند. این به معنای آن بود که یک توکن دزدیده‌شده می‌توانست بارها برای دریافت توکن دسترسی جدید استفاده شود.

۲.۳ راه‌حل: Rotation Token

الگوریتم چرخش توکن تازه‌سازی پیاده‌سازی شد:

۱. کاربر توکن تازه‌سازی را به `POST /api/auth/refresh` ارسال می‌کند
۲. سرور توکن قدیمی را در پایگاه داده پیدا می‌کند
۳. اگر توکن قبلاً باطل شده باشد، تمام توکن‌های آن کاربر باطل می‌شوند (احتمال سرقت توکن)
۴. توکن قدیمی با `IsRevoked = true` علامت‌گذاری می‌شود
۵. توکن دسترسی جدید + توکن تازه‌سازی جدید صادر می‌شود

۳.۳ پیاده‌سازی

کلاس `RefreshTokenCommandHandler` این منطق را پیاده‌سازی می‌کند. موجودیت `RefreshToken` در پایگاه داده با فیلدهای `IsRevoked` و `RevokedAt` نگهداری می‌شود.

۴ محدودیت نرخ درخواست‌ها

۱.۴ پیاده‌سازی

محدودیت نرخ با ASP.NET Core Rate Limiting Middleware پیاده‌سازی شد. دو پروفایل تعریف شدند:

نام پروفایل	حداکثر درخواست	بازه زمانی	کاربرد
auth	۵ درخواست	۱ دقیقه	Endpoint های احراز هویت
general	۱۰۰ درخواست	۱ دقیقه	سایر Endpoint ها

۲.۴ رفتار در صورت تجاوز

پس از تجاوز از حد مجاز، پاسخ Too Many Requests 429 با پیام فارسی برگردانده می‌شود:

```
{
  "Success": false,
  "Message": "تجاوز از حد مجاز",
  "RetryAfter": "60 seconds"
}
```

۳.۴ هدف امنیتی

پروفایل auth جلوگیری از:

- حمله Brute Force برای OTP
- حمله Credential Stuffing برای ورود با رمز عبور
- OTP Bombing (ارسال انبوه OTP به یک شماره)

۵ رفع مشکل CORS برای پورت ۵۱۷۴

۱.۵ مشکل

Vite در برخی شرایط (وقتی پورت ۵۱۷۳ اشغال است) روی پورت ۵۱۷۴ اجرا می‌شود. پیکربندی CORS اولیه فقط `http://localhost:5173` را مجاز می‌دانست.

۲.۵ راه‌حل

پورت ۵۱۷۴ به لیست AllowedOrigins در `appsettings.json` اضافه شد:

```
"AllowedOrigins": [
  "http://localhost:5173",
  "http://localhost:5174"
]
```

۶ استانداردسازی پاسخ خطا با RFC ۷۸۰۷

۱.۶ مشکل

پاسخ‌های خطا فرمت یکسانی نداشتند. برخی اوقات رشته متن ساده، برخی اوقات JSON بدون ساختار مشخص برگردانده می‌شد.

۲.۶ راه‌حل

ExceptionHandlingMiddleware برای رهگیری تمام خطاها و برگرداندن پاسخ استاندارد - Prob-lem Details (RFC 7807) پیاده‌سازی شد:

```
{
  "type": "https://tools.ietf.org/html/rfc7807",
  "title": "Not Found",
  "status": 404,
  "detail": " ",
  "instance": "/api/reservations/abc-123"
}
```

۳.۶ کدهای وضعیت استاندارد

کد	نام	کاربرد
400	Bad Request	خطای اعتبارسنجی (FluentValidation)
401	Unauthorized	توکن نامعتبر یا منقضی
403	Forbidden	عدم دسترسی بر اساس نقش
404	Not Found	موجودیت یافت نشد
409	Conflict	تداخل (مثل رزرو همزمان یک میز)
429	Too Many Requests	تجاوز از محدودیت نرخ
500	Internal Server Error	خطای غیرمنتظره

۷ بهبود امنیت OTP

۱.۷ یکبار مصرف بودن OTP

در پیاده‌سازی اولیه، کد OTP پس از تأیید از Redis حذف نمی‌شد. این باگ رفع شد: پس از تأیید موفق کد، IDatabase.KeyDelete فراخوانی می‌شود.

۲.۷ حذف OTP از لاگ

لاگ‌های Serilog حاوی کد OTP بودند که خطر امنیتی ایجاد می‌کرد. تمام log statement‌هایی که مقدار OTP را ثبت می‌کردند حذف شدند. به جای آن، یک لاگ با فرمت OTP sent to {phone[..3]}*** جایگزین شد که شماره موبایل کامل را نشان نمی‌دهد.

۸ رفع باگ‌های فرانت‌اند

۱.۸ باگ ۱: مدیریت خطای ۴۰۱ در client.ts

مشکل: هنگامی که توکن دسترسی منقضی می‌شد، API پاسخ 401 برمی‌گرداند. فرانت‌اند این پاسخ را مدیریت نمی‌کرد و کاربر یک صفحه خطای خام می‌دید.

راه‌حل: در client.ts (لایه API) یک رهگیر (interceptor) اضافه شد که پاسخ‌های 401 را شناسایی کرده، توکن از localStorage حذف می‌شود و کاربر به صفحه ورود هدایت می‌شود.

۲.۸ باگ ۲: پیام‌های خطای انگلیسی

مشکل: پیام‌های خطا از API به انگلیسی بودند و مستقیم نمایش داده می‌شدند.

راه‌حل: نقشه‌برداری از کدهای خطا به پیام‌های فارسی در client.ts اضافه شد. همچنین بک‌اند برای برخی خطاهای رایج پیام فارسی مستقیم برمی‌گرداند.

۳.۸ باگ ۳: تقویم شمسی در فرم رزرو

مشکل: تاریخ انتخاب‌شده از تقویم شمسی به درستی به فرمت DateOnly که بک‌اند انتظار داشت تبدیل نمی‌شد.

راه‌حل: تابع تبدیل تاریخ با استفاده از کتابخانه date-fns اصلاح شد. تاریخ شمسی ابتدا به میلادی تبدیل سپس به فرمت YYYY-MM-DD فرمت‌بندی می‌شود.

۴.۸ باگ ۴: HasIncompleteProfile (تغییر نام IsNewUser)

مشکل: فیلد IsNewUser در پاسخ verify-otp نام گمراه‌کننده‌ای داشت. این فیلد نشان می‌دهد که کاربر باید پروفایل خود را تکمیل کند، نه اینکه حتماً کاربر جدید است.

راه‌حل: نام فیلد به HasIncompleteProfile تغییر یافت. فرانت‌اند بر اساس این فیلد کاربر را به /complete-profile هدایت می‌کند.

۹ بهبود امنیت ورود مدیر ارشد

۱.۹ مشکل

صفحه ورود مدیر ارشد (/admin/login) از همان فرم ورود با OTP استفاده می‌کرد که برای کاربران عادی طراحی شده بود.

۲.۹ راه‌حل

یک Endpoint مستقل POST /api/admin/auth/login با AdminLoginCommand ایجاد شد که:

- فقط با نام کاربری و رمز عبور کار می‌کند (نه OTP)
- رمز عبور با BCrypt تأیید می‌شود
- توکن با نقش Admin صادر می‌شود
- صفحه ورود ادمین در فرانت‌اند طراحی مستقل دارد

۱۰ صفحات تکمیل شده در این اسپرینت

۱.۱۰ صفحه تکمیل پروفایل (CompleteProfile)

صفحه `/complete-profile` برای کاربرانی که پس از ورود با OTP پروفایل ناقص دارند نمایش داده می شود:

- وارد کردن نام نمایشی (اجباری)
- آپلود تصویر پروفایل (اختیاری)
- این صفحه توسط ProtectedRoute با نقش customer محافظت می شود
- پس از تکمیل، به داشبورد مشتری هدایت می شود

۲.۱۰ پروفایل مشتری (Profile)

صفحه `/customer-dashboard/profile`:

- نمایش اطلاعات فعلی کاربر
- ویرایش نام نمایشی
- تغییر تصویر پروفایل با آپلود
- تغییر رمز عبور (فرم جداگانه)
- نمایش شماره موبایل (فقط خواندنی)

۳.۱۰ پروفایل مدیر رستوران (ManagerProfile)

مشابه پروفایل مشتری با امکانات مشابه.

۴.۱۰ مدیریت میز (TableManagement)

صفحه مدیریت میز از اسپرینت قبل تکمیل شد:

- لیست میزهای فعلی با شماره و ظرفیت
- فرم افزودن میز جدید
- دکمه حذف با دیالوگ تأیید
- نمایش وضعیت اشغال بودن میز

۱۱ تست نهایی و استقرار

۱.۱۱ تست‌های انجام‌شده

سناریو	نتیجه	توضیح
جریان کامل ثبت‌نام مشتری	موفق	OTP + تکمیل پروفایل + داشبورد
جریان کامل ورود مشتری	موفق	OTP + هدایت به داشبورد
ثبت رستوران توسط مدیر	موفق	فرم چندمرحله‌ای + ارسال برای تأیید
تأیید رستوران توسط ادمین	موفق	تغییر وضعیت به Approved
رزرو کامل توسط مشتری	موفق	انتخاب تاریخ، میز، ارسال
تأیید رزرو توسط مدیر رستوران	موفق	تغییر وضعیت + اعلان
لغو رزرو توسط مشتری	موفق	تغییر وضعیت به Cancelled
محدودیت نرخ OTP	موفق	بعد از ۵ درخواست، 429
انقضا توکن و تجدید	موفق	Token Rotation کار می‌کند
ورود ادمین	موفق	دسترسی به پنل ادمین

۲.۱۱ محیط استقرار

- بک‌اند: ASP.NET Core 8 روی Docker
- پایگاه داده: PostgreSQL 16 روی Docker
- کش: Redis 7 روی Docker
- فرانت‌اند: Vite dev server برای توسعه
- Docker Compose برای اجرای همه سرویس‌ها با یک دستور

۱۲ بازبینی و گذر اسپرینت

۱.۱۲ Review Sprint

- تمامی اهداف امنیتی محقق شدند
- تمام باگ‌های شناسایی شده رفع شدند
- سیستم آماده تحویل نهایی است
- جریان کامل سیستم (از ثبت نام تا رزرو و مدیریت) بدون خطا کار می‌کند

۲.۱۲ Retrospective Sprint

چه چیزی خوب بود:

- تمرکز بر کیفیت و امنیت به جای اضافه کردن ویژگی‌های جدید
 - استفاده از استاندارد RFC 7807 فرمت پاسخ‌های خطا را یکدست کرد
 - Token Rotation امنیت سیستم را به طور قابل توجهی بهبود داد
- درس‌های آموخته شده از کل پروژه:
- معماری تمیز از ابتدا هزینه توسعه را در درازمدت کاهش می‌دهد
 - CQRS + MediatR کدهای Business Logic را از Controller جدا و تمیز نگه می‌دارد
 - استفاده از Redis برای OTP بسیار سریع تر و مناسب تر از پایگاه داده است
 - Rate Limiting باید از همان اسپرینت اول پیاده سازی شود

۳.۱۲ آمار کلی پروژه

مقدار	معیار
بیش از ۳۵	تعداد Endpoint های API
۹ موجودیت	تعداد موجودیت های دامنه
بیش از ۴۰ کامند/کوئری	تعداد Feature های CQRS
۲۵ صفحه	تعداد صفحات فرانت اند
بیش از ۱۵ کامپوننت	تعداد کامپوننت های مشترک
۳ اسپرینت	تعداد اسپرینت
۶ هفته	مدت کل پروژه